

SwarmOps Cloud

Database Design

Nima Sarayan

September 2026

Abstract

SwarmOps Cloud keeps all of its state in one relational database, MariaDB 11.4 or MySQL 8.4. This document presents the design of that database. It moves from a conceptual model of the business, through the logical schema drawn as entity-relationship diagrams, to the physical choices of types, keys, constraints and indexes. It argues that the schema is in third normal form and names each deliberate exception with its reason. It explains how transactions, isolation and row locks prevent the concurrency anomalies that would otherwise corrupt money, and it shows real query plans. The schema has 57 tables and 3 views, created by nine migrations; a complete data dictionary generated from the live schema is given as an appendix.

Contents

1	Introduction	3
1.1	Requirements on the data	3
2	Conceptual model	4
3	Logical model	4
4	Physical design	4
4.1	Engine, character set and time	4
4.2	Types	4
4.3	Migrations	7
4.4	Naming conventions	8
5	Normalisation	8
5.1	First normal form	8
5.2	Second normal form	8
5.3	Third normal form and BCNF	8
5.4	Deliberate denormalisation	9
6	Integrity constraints	9
6.1	Keys	9
6.2	Foreign keys	10
6.3	CHECK constraints	10
7	Indexes and query plans	11
8	Transactions, isolation and concurrency	11
8.1	Isolation level	11
8.2	Anomalies prevented	11
9	Views	12
10	Security of stored data	13
11	Sample queries	14
12	Backup, restore and upgrade	14
13	Appendix: data dictionary	14
13.1	Commerce tables	14
13.1.1	users	14
13.1.2	user_sessions	15
13.1.3	product_categories	15
13.1.4	plans	15
13.1.5	plan_features	16
13.1.6	wallets	16
13.1.7	wallet_transactions	17
13.1.8	orders	17
13.1.9	order_items	18
13.1.10	projects	18
13.1.11	usage_records	19
13.1.12	invoices	20

13.1.13	invoice_lines	20
13.1.14	support_tickets	20
13.1.15	ticket_messages	21
13.2	Platform tables	21
13.2.1	agents	21
13.2.2	agent_ca	21
13.2.3	agent_claims	22
13.2.4	agent_enrollment_tokens	22
13.2.5	applications	22
13.2.6	application_databases	23
13.2.7	application_database_credentials	23
13.2.8	application_database_env	23
13.2.9	application_env	24
13.2.10	application_health_command	24
13.2.11	application_outcomes	24
13.2.12	audit_events	25
13.2.13	audit_event_details	25
13.2.14	commands	25
13.2.15	command_events	26
13.2.16	command_locks	26
13.2.17	command_outputs	26
13.2.18	command_payloads	27
13.2.19	core_authority	27
13.2.20	core_handoffs	27
13.2.21	core_members	28
13.2.22	database_credentials	28
13.2.23	dependency_bindings	28
13.2.24	dns_credential_versions	28
13.2.25	dns_records	29
13.2.26	routes	29
13.2.27	route_certificates	30
13.2.28	route_certificate_domains	31
13.2.29	route_hosts	31
13.2.30	route_runtime	31
13.2.31	route_runtime_entry_points	32
13.2.32	route_runtime_errors	32
13.2.33	routing_clusters	32
13.2.34	routing_domains	33
13.2.35	schema_migrations	33
13.2.36	servers	33
13.2.37	server_agent_events	34
13.2.38	server_keys	34
13.2.39	service_route_declarations	35
13.2.40	source_connections	35
13.2.41	source_private_hosts	35
13.2.42	source_settings	35

List of Figures

1	Commerce schema	5
2	Platform schema: keys and relationships	6

List of Tables

1	Requirements on the data	3
2	Choice of column types	7
3	Migrations	7
4	Deliberate denormalisation	9
5	Foreign-key rules in the commerce schema	10
6	Examples of CHECK constraints	10
7	Indexes of the most frequent queries	11
8	Concurrency anomalies and their prevention	12

1 Introduction

The database serves two purposes. It replaces the encrypted snapshot files in which SwarmOps kept its controller state (servers, applications, routing, the command queue, the audit ledger and others). It also holds the new commerce data: customers, plans, wallets, orders, projects, usage, invoices and support tickets. Keeping both in one database is the design’s central decision: confirming an order must charge a wallet and queue a deployment in a single transaction (Software Architecture Document, section 6.1).

1.1 Requirements on the data

Table 1: Requirements on the data

ID	Requirement	Source
D-1	A wallet balance is never negative and always equals the sum of its ledger.	FR-WAL-1, FR-WAL-3, NFR-REL-1
D-2	A retried top-up, charge or refund is applied at most once.	FR-WAL-2, FR-PRV-2
D-3	A project is charged at most once per hour, and its charges in a month never exceed its plan’s monthly cap.	FR-BIL-1, FR-BIL-2
D-4	Confirmation, charge and deployment request commit together.	FR-ORD-6
D-5	Every reference points at an existing row.	NFR-REL-3
D-6	Secrets are unreadable without the separately held data key.	FR-DAT-4
D-7	The schema runs unchanged on MariaDB 11.4 and MySQL 8.4.	NFR-PORT-1
D-8	Reports are available without application code.	FR-REP-1, FR-REP-2

2 Conceptual model

The commerce domain has these entities and relationships:

- A **user** is a customer or an administrator. Each customer has exactly one **wallet**, and every change to it is a **wallet transaction**.
- A **plan** belongs to a **product category** and lists **features**.
- A customer places **orders**. Each order has one or more **order items**, each naming a plan together with the application to run and the prices at the time of ordering. Administrators review orders.
- A confirmed order item produces one **project**. A project accrues one **usage record** for each charged hour, and each usage record is paid by one wallet transaction.
- Each month a customer receives an **invoice**. Its **invoice lines** summarise the usage records it covers, and each usage record is covered by at most one invoice.
- A customer opens **support tickets**, which contain **ticket messages** written by the customer or by administrators.

The platform domain consists of groups that existed before as separate snapshot files:

- **audit**: events and their details;
- **applications**: environment, databases, health commands and outcomes, plus credentials;
- **source**: connections, settings and private hosts;
- **servers**: keys and agent events;
- **core topology**: authority, members and handoffs;
- **agent registry**: the certificate authority, agents, tokens and claims;
- **routing**: clusters, routes and their hosts, DNS, certificates and runtime observations;
- **the command queue**: commands, payloads, events, outputs and locks.

A deployment connects the two domains. A project's `command_id` names the `application.deploy` command that provisioned it, and its `app_name` is the name of the application that command created.

3 Logical model

Figure 1 shows the commerce schema with every column. Figure 2 shows the platform schema's tables with their keys and relationships. Crow's-foot notation is used: a bar means *exactly one*, a circle *zero*, and a fork *many*.

Both diagrams are generated from the live schema by `docs/academic/tools/schema_docs.py`. They cannot drift from the migrations.

4 Physical design

4.1 Engine, character set and time

Every table uses InnoDB, the transactional engine of both MariaDB and MySQL, with `utf8mb4` and `utf8mb4_unicode_ci`. This stores Persian text, and any other script, correctly. The connection forces UTC and `parseTime`. All timestamps are `DATETIME(6)` in UTC with microsecond precision, except hour boundaries in `usage_records.period_start`, which are whole-second `DATETIME`, and invoice periods, which are `DATE`.

4.2 Types

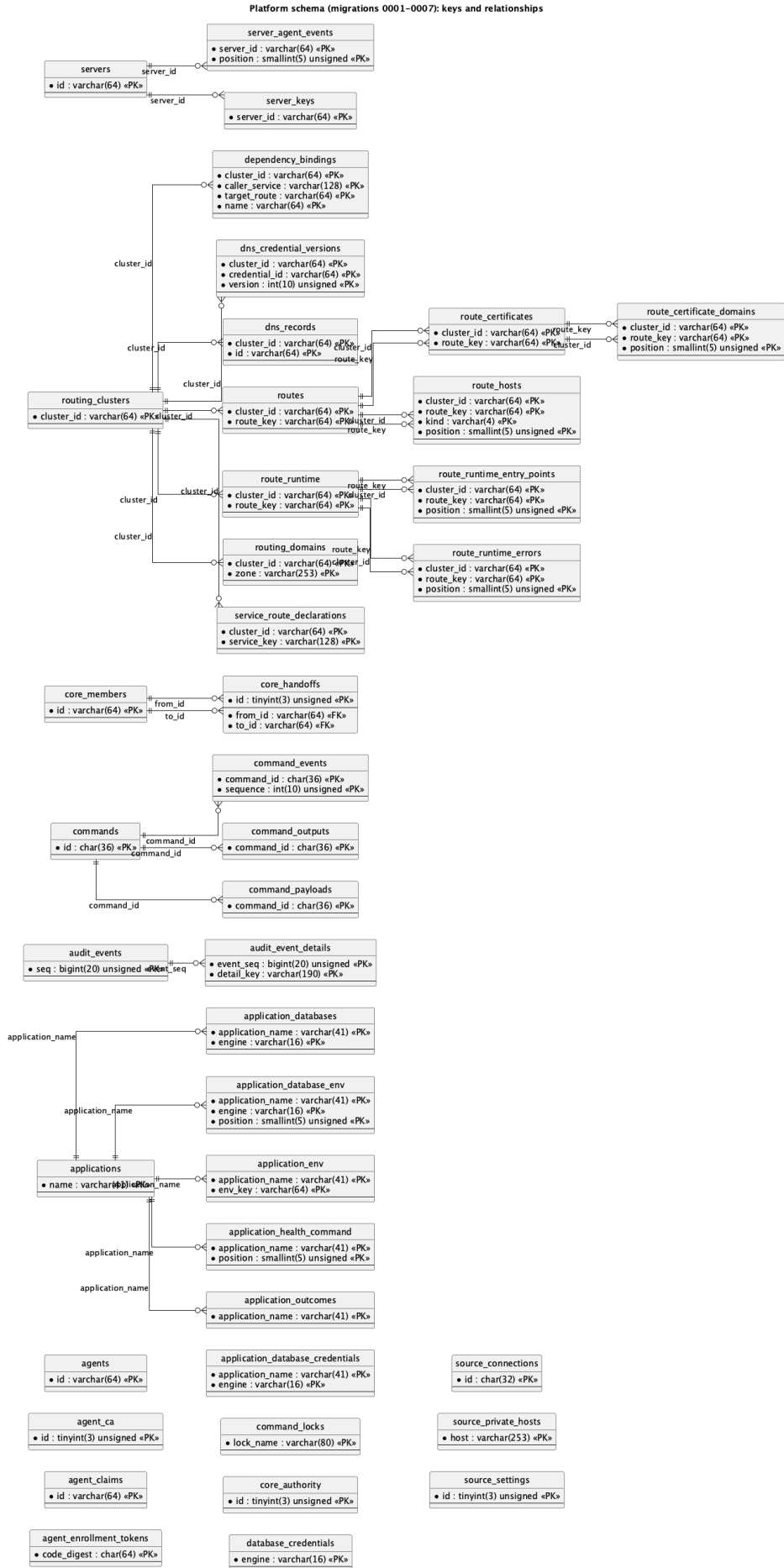


Figure 2: Platform schema: keys and relationships

Table 2: Choice of column types

Kind of value	Type	Reason
Money	BIGINT (signed for ledger amounts, unsigned for prices)	Exact integer rials; no rounding error. The largest top-up (2×10^1 rials) is far below the 9.2×10^1 limit.
Surrogate keys	BIGINT UNSIGNED AUTO_INCREMENT	Never reused; ample range.
Command identifiers	CHAR(36)	Identifiers generated by the application, so a command's id is known before it is inserted.
Session tokens	CHAR(64)	The SHA-256 hash of the token, in hex; the token itself is never stored.
Statuses and kinds	VARCHAR(16) with a CHECK constraint	Portable between the two engines and extensible by migration; the CHECK constraint gives the same protection an ENUM would. See section 5.1.
Whole routing documents	JSON (MariaDB stores it as LONGTEXT checked with JSON_VALID)	
Secrets	VARBINARY / BLOB holding AES-GCM ciphertext	See section 9.

4.3 Migrations

The schema is created by nine SQL files embedded in the binary and applied in order at start-up. Section 6.5 of the Software Architecture Document describes the runner.

Table 3: Migrations

Migration	Tables	Content
0001_audit.sql	2	Audit events and details
0002_applications.sql	8	Applications, environment, health commands, databases, outcomes, credentials
0003_source.sql	3	Source connections, settings, private hosts
0004_servers.sql	3	Servers, agent events, keys
0005_core_and_agents.sql	7	Core authority, members, handoffs; agent CA, agents, tokens, claims
0006_routing.sql	13	Routing clusters, routes, hosts, bindings, declarations, domains, DNS, certificates, runtime
0007_command_queue.sql	5	Commands, payloads, events, outputs, locks

Migration	Tables	Content
0008_commerce.sql	15 + 3 views	The commerce schema
0009_plan_text.sql	0	Persian plan descriptions and features; corrected seeded features
<i>runner</i>	1	schema_migrations
Total	57 + 3 views	

4.4 Naming conventions

Constraint names state their kind and table: `uq_` for unique keys, `ix_` for secondary indexes, `fk_` for foreign keys and `ck_` for CHECK constraints. Examples are `uq_usage_records_hour`, `ix_orders_queue`, `fk_wallet_transactions_wallet` and `ck_wallets_non_negative`. Error messages therefore name the rule that was broken.

5 Normalisation

5.1 First normal form

Every column holds one atomic value, and there are no repeating groups. Values that could have been lists are separate tables, each row ordered where order matters: a plan's features (`plan_features`), an application's environment variables (`application_env`), the arguments of a health command (`application_health_command`), a route's host names (`route_hosts`) and a certificate's domains (`route_certificate_domains`).

The exception. `routing_clusters.settings_json`, `cutover_json` and `cutover_rollback_json` hold whole JSON documents: a cluster's gateway settings, its cutover plan and that plan's rollback. These are always read, validated and written as a unit by the routing store, under a lock on the cluster row, and no query ever filters on a field inside them. Splitting them into rows would add many tables that are never queried, and would move document validation from one place into many. The routes, domains, DNS records and certificates inside a cluster are ordinary normalised tables, because they *are* queried individually.

5.2 Second normal form

Almost every table has a single-column key, so no partial dependency can exist. The tables with composite keys have attributes that depend on the whole key:

- `plan_features(plan_id, locale, position)`: a feature is the feature at *that* position of *that* plan in *that* language.
- `application_env(application, name)`: a value belongs to *that* variable of *that* application.

5.3 Third normal form and BCNF

No non-key attribute depends on another non-key attribute. Three designs look like transitive dependencies but are not:

- **Prices in `order_items` and `projects`.** The hourly and monthly prices are copied from the plan at the time of ordering. They do not depend on `plan_id`: they record what the customer agreed to pay, which must stay the same after the plan's price changes (FR-CAT-4). The plan's current price and the order's agreed price are different facts.

- **orders.user_id and projects.user_id.** A project’s owner could be reached through its order item and order. It is stored on the project because every customer query is scoped by **user_id** (NFR-SEC-2). The value is written once, when the project is created in the confirmation transaction, and never changes.
- **invoice_lines.project_id and the line’s amount.** An issued invoice is a historical document. Its lines must not change if usage records are later altered, so they store the amounts that were invoiced.

Every determinant in the commerce tables is a candidate key. In **users**, for example, **email** is unique, and **id** and **email** are the only determinants. The commerce schema is therefore also in Boyce–Codd normal form.

5.4 Deliberate denormalisation

Table 4: Deliberate denormalisation

Stored value	Derivable from	Why it is stored	How consistency is kept
wallets.balance_rial	SUM(wallet_transactions.amount_rial)	The amount_rial) overdraft checks must read one row under a lock, rather than sum a growing ledger.	Updated in the same transaction as the ledger row, under FOR UPDATE on the wallet; CHECK (balance_rial >= 0); the console verifies it equals the ledger.
wallet_transactions.balance_after_rial	Balance after earlier rows	Shows each row’s resulting balance without a window function over the whole ledger.	Written by the same code, in the same transaction; CHECK (balance_after_rial >= 0).
invoices.subtotal_rial, tax_rial, total_rial	Sum of the lines and the tax formula	An issued invoice is fixed; recomputing it later could change a legal document.	CHECK (total_rial = subtotal_rial + tax_rial).
servers agent-health columns	The latest server_agent_events row	The console lists servers with their health in one query.	Updated together with the event insert.

6 Integrity constraints

6.1 Keys

Every table has a primary key. Unique keys express business rules:

- users.email and plans.code;
- projects.app_name, which resolves a race between two orders for the same name;
- invoices.number;
- wallet_transactions(user_id, idempotency_key);
- usage_records(project_id, period_start);
- commands(actor, idempotency_key).

6.2 Foreign keys

There are 48 foreign keys: 18 RESTRICT, 28 CASCADE and 2 SET NULL. A foreign key without an explicit ON DELETE clause behaves as RESTRICT; the data dictionary lists the effective rule of each.

The commerce schema chooses each rule by what the reference means:

Table 5: Foreign-key rules in the commerce schema

Rule	References	Reason
RESTRICT	wallets, ledger rows, orders, order items, projects, usage records and invoices to their users, plans, wallets, projects and invoices	Money and history refer to these rows, so a referenced row cannot be deleted.
CASCADE	user → sessions, plan → features, order → order items, invoice → invoice lines, ticket → messages	The child has no meaning without its parent.
SET NULL	invoice_lines.project_id, support_tickets.project_id	An invoice line or a ticket must survive even if its project row were removed; it keeps its own description.

Most of the platform schema’s CASCADE rules delete detail rows with their parent, such as a server’s keys and agent events, an application’s environment variables, and a command’s events and outputs.

6.3 CHECK constraints

There are 42 CHECK constraints. They include:

Table 6: Examples of CHECK constraints

Constraint	Rule
ck_wallets_non_negative	balance_rial >= 0
ck_wallet_transactions_kind	kind IN ('topup','order_charge','usage','refund','adjustmen
ck_wallet_transactions_amount	amount_rial <> 0
ck_wallet_transactions_balance	balance_after_rial >= 0
ck_invoices_total	total_rial = subtotal_rial + tax_rial
ck_orders_review	status IN ('pending','cancelled') OR reviewed_at IS NOT NULL: a confirmed, rejected or failed order always records when it was reviewed
ck_plan_features_locale	locale IN ('en','fa')
ck_commands_state	state IN ('uploading','queued','leased','preparing','running
ck_agent_ca_singleton, source_settings	id = 1, so the table has exactly one row
ck_applications_resources	cpus > 0 AND memory_mib > 0 AND port > 0

MySQL reports a violated CHECK constraint as error 3819 and MariaDB as error 4025. `sqlstore.IsCheckViolation` recognises both.

7 Indexes and query plans

Every foreign key is indexed, and each frequent query has an index that matches its filter and order. The plans below were produced with `EXPLAIN` on MariaDB 11.4 against the development database used for the end-to-end run. That database is small, so the optimiser’s choices for tiny tables are noted where they differ from what larger tables would use.

Table 7: Indexes of the most frequent queries

Query	Index	Plan observed
Claim the next due command: WHERE state IN (‘queued’, ‘retry_scheduled’) AND (next_attempt_at IS NULL OR next_attempt_at ≤ now) ORDER BY created_at, seq LIMIT 1 FOR UPDATE SKIP LOCKED	<code>ix_commands_runnable</code>	<code>range on</code> <code>ix_commands_runnable</code> , using the index condition, then a filesort of the few runnable rows
Sum a project’s usage in a month: WHERE project_id = ? AND period_start ≥ ? AND period_start < ?	<code>uq_usage_records_hour</code> (project_id, period_start)	<code>range on</code> <code>uq_usage_records_hour</code>
Pending-order queue: WHERE status = ‘pending’ ORDER BY placed_at	<code>ix_orders_queue</code> (status, placed_at)	<code>ref on ix_orders_queue</code> , <i>Using index</i> (covering; no sort)
A customer’s ledger: WHERE user_id = ? ORDER BY id DESC LIMIT 50	<code>ix_wallet_transactions_user</code> (user_id, created_at)	On six rows the optimiser scanned the primary key; with a realistic ledger the <code>user_id</code> index is selective

The same `uq_usage_records_hour` key has two roles. It is the index for the monthly sum, and the constraint that makes a second charge for the same hour impossible.

8 Transactions, isolation and concurrency

8.1 Isolation level

All application transactions run at **READ COMMITTED**. Every read whose result decides a write is made with `SELECT ... FOR UPDATE`, so the protection REPEATABLE READ’s consistent snapshot would give is obtained explicitly. **READ COMMITTED** also avoids the gap locks REPEATABLE READ takes on range reads, which would make unrelated inserts wait for each other — usage rows for different projects, for example. A transaction the server aborts to break a deadlock (error 1213) or a lock wait (1205) is repeated as a whole, up to four attempts.

8.2 Anomalies prevented

Table 8: Concurrency anomalies and their prevention

Anomaly	Where it could happen	Prevention
Lost update	Two charges read the same balance and both write back.	The wallet row is locked with <code>FOR UPDATE</code> before reading the balance.
Overdraft under concurrency	Fifty parallel charges exceed the balance.	The row lock serialises them, and <code>ck_wallets_non_negative</code> rejects any that would still overdraw. The test's result: 33 succeed, 17 are refused, and the balance equals the ledger.
Duplicate application of a retry	A client repeats a top-up after a timeout.	<code>UNIQUE(user_id, idempotency_key)</code> ; the second attempt returns the first transaction.
Double billing of an hour	Two billing runs overlap.	<code>UNIQUE(project_id, period_start)</code> on <code>usage_records</code> .
Double confirmation	Two administrators confirm the same order.	The order row is locked and must still be <code>pending</code> ; the second confirmation receives a conflict.
Charge without deployment	The process stops between charging and queuing.	Both happen in one transaction (transactional outbox).
Double claim of a command	Two workers claim the same command.	<code>FOR UPDATE SKIP LOCKED</code> on the claim.
Invoice missing an hour	Billing adds an hour between summing and marking usage invoiced.	Invoicing locks the uninvoiced usage rows with <code>FOR UPDATE</code> before summing them.
Concurrent migrations	Two cores start at once.	<code>GET_LOCK('swarmops_schema_migrations', 60)</code> .
Two active owners	Two cores both believe they are active.	The singleton <code>core_authority</code> row, locked <code>FOR UPDATE</code> , holds the authority epoch.

9 Views

Three views give administrators reports without application code (D-8):

```
CREATE VIEW v_customer_accounts AS
SELECT u.id AS user_id, u.email, u.full_name, u.status, u.created_at,
       COALESCE(w.balance_rial, 0) AS balance_rial,
       (SELECT COUNT(*) FROM projects p WHERE p.user_id = u.id
        AND p.status IN ('provisioning', 'active', 'suspended')) AS live_projects,
       (SELECT COUNT(*) FROM orders o WHERE o.user_id = u.id
        AND o.status = 'pending') AS pending_orders
```

```

FROM users u
LEFT JOIN wallets w ON w.user_id = u.id
WHERE u.role = 'customer';

CREATE VIEW v_order_queue AS
SELECT o.id AS order_id, o.placed_at, u.id AS user_id, u.email,
       COALESCE(w.balance_rial, 0) AS balance_rial, i.app_name,
       pl.code AS plan_code, pl.name AS plan_name,
       i.hourly_price_rial, i.monthly_price_rial
FROM orders o
JOIN users u ON u.id = o.user_id
JOIN order_items i ON i.order_id = o.id
JOIN plans pl ON pl.id = i.plan_id
LEFT JOIN wallets w ON w.user_id = o.user_id
WHERE o.status = 'pending';

CREATE VIEW v_revenue_by_month AS
SELECT DATE_FORMAT(t.created_at, '%Y-%m') AS month,
       SUM(CASE WHEN t.kind IN ('order_charge', 'usage')
            THEN -t.amount_rial ELSE 0 END) AS charged_rial,
       SUM(CASE WHEN t.kind = 'refund' THEN t.amount_rial ELSE 0 END) AS refunded_rial,
       SUM(CASE WHEN t.kind = 'topup' THEN t.amount_rial ELSE 0 END) AS topped_up_rial,
       COUNT(DISTINCT t.user_id) AS paying_customers
FROM wallet_transactions t
GROUP BY DATE_FORMAT(t.created_at, '%Y-%m');

```

10 Security of stored data

The columns that hold secrets store AES-GCM ciphertext:

- application environment values;
- database credentials;
- source-connection tokens and the source settings password;
- DNS provider secrets;
- server keys;
- the agent certificate authority's private key;
- command payloads and outputs.

The encryption key is read from a protected file and is never written to the database (D-6). Each ciphertext is bound to its location by its associated data, `swarmops-sql:<table>.<column>:<key values>`. A value copied into another row or column fails authentication instead of decrypting. Anyone who has a database backup but not the key therefore sees neither the secrets nor a way to move them between rows.

Passwords are never stored: `users.password_hash` holds a bcrypt hash. Storefront session tokens are stored only as SHA-256 hashes.

Recommendation for production. Run migrations with a database account that may change the schema, and run Core with an account limited to data manipulation on the `swarmops` schema.

11 Sample queries

Wallets whose balance differs from the sum of their ledger. The expected result is empty, and it was empty on the development database after the end-to-end run:

```
SELECT w.user_id, w.balance_rial, COALESCE(SUM(t.amount_rial), 0) AS ledger_sum
FROM wallets w
LEFT JOIN wallet_transactions t ON t.user_id = w.user_id
GROUP BY w.user_id, w.balance_rial
HAVING w.balance_rial <> COALESCE(SUM(t.amount_rial), 0);
```

Customers with the most usage this month:

```
SELECT p.user_id, u.email, SUM(r.amount_rial) AS usage_rial
FROM usage_records r
JOIN projects p ON p.id = r.project_id
JOIN users u ON u.id = p.user_id
WHERE r.period_start >= DATE_FORMAT(UTC_TIMESTAMP(), '%Y-%m-01')
GROUP BY p.user_id, u.email
ORDER BY usage_rial DESC
LIMIT 10;
```

Projects that a failed deployment left without a running service:

```
SELECT p.app_name, p.status, c.state, c.failure_code, c.failure_summary
FROM projects p
JOIN commands c ON c.id = p.command_id
WHERE p.status = 'failed';
```

12 Backup, restore and upgrade

- **Backup:** `mysqldump --single-transaction --routines --triggers swarmops`. This takes a consistent InnoDB snapshot without locking writers. The backup must be kept together with a separately protected copy of the data key; either one alone is insufficient.
- **Restore:** load the dump into an empty schema and start Core with the same key. The migration runner verifies the recorded checksums and applies only newer migrations.
- **Upgrade from snapshot files:** `swarmops-core migrate-state` imports each former snapshot store into its tables, one transaction per store. It leaves the files in place as a backup.

13 Appendix: data dictionary

The data dictionary that follows is generated from the live schema. For each table it gives the table's purpose, and for each column its type, whether it may be null, its key and its default. It then lists the table's foreign keys, each with its ON DELETE rule, and its CHECK constraints.

Generated from the live schema created by migrations 0001 to 0009. Key: PRI primary, UNI unique, MUL indexed.

13.1 Commerce tables

13.1.1 users

Customer and administrator accounts with bcrypt password hashes.

Column	Type	Null	Key	Default	References
id	bigint(20) unsigned	no	PRI	auto_increment	
email	varchar(254)	no	UNI		
full_name	varchar(120)	no			
password_hash	varchar(100)	no			
role	varchar(16)	no	MUL		
status	varchar(16)	no		'active'	
created_at	datetime(6)	no			
updated_at	datetime(6)	no			
last_login_at	datetime(6)	yes			

Check constraints:

- ck_users_role: role in ('customer','admin')
- ck_users_status: status in ('active','suspended')

13.1.2 user_sessions

Storefront sessions: token hash, CSRF token, expiry and revocation.

Column	Type	Null	Key	Default	References
token_hash	char(64)	no	PRI		
user_id	bigint(20) unsigned	no	MUL		users.id
csrf_token	varchar(64)	no			
created_at	datetime(6)	no			
expires_at	datetime(6)	no			
revoked_at	datetime(6)	yes			

Foreign keys:

- fk_user_sessions_user → users (On delete CASCADE)

13.1.3 product_categories

Groups of plans shown in the storefront.

Column	Type	Null	Key	Default	References
id	smallint(5) unsigned	no	PRI	auto_increment	
code	varchar(32)	no	UNI		
name	varchar(80)	no			
description	varchar(255)	no			

13.1.4 plans

The products: CPU, memory, disk, hourly price and monthly cap.

Column	Type	Null	Key	Default	References
id	int(10) unsigned	no	PRI	auto_increment	

Column	Type	Null	Key	Default	References
category_id	smallint(5) unsigned	no	MUL		product_categories.id
code	varchar(32)	no	UNI		
name	varchar(64)	no			
description	varchar(255)	no			
description_fa	varchar(255)	no		''	
cpu_millicores	int(10) unsigned	no			
memory_mib	int(10) unsigned	no			
disk_gib	int(10) unsigned	no			
hourly_price_rial	int(20) unsigned	no			
monthly_price_rial	int(20) unsigned	no			
is_active	tinyint(1)	no	MUL	1	
sort_order	smallint(5) unsigned	no		0	
created_at	datetime(6)	no			
updated_at	datetime(6)	no			

Foreign keys:

- fk_plans_category → product_categories (On delete RESTRICT)

Check constraints:

- ck_plans_prices: hourly_price_rial > 0 and monthly_price_rial >= hourly_price_rial
- ck_plans_resources: cpu_millicores > 0 and memory_mib > 0

13.1.5 plan_features

Marketing features listed for a plan.

Column	Type	Null	Key	Default	References
plan_id	int(10) unsigned	no	PRI		plans.id
locale	varchar(8)	no	PRI	'en'	
position	smallint(5) unsigned	no	PRI		
feature	varchar(160)	no			

Foreign keys:

- fk_plan_features_plan → plans (On delete CASCADE)

Check constraints:

- ck_plan_features_locale: locale in ('en','fa')

13.1.6 wallets

One prepaid wallet per user; the cached balance is never negative.

Column	Type	Null	Key	Default	References
user_id	bigint(20) unsigned	no	PRI		users.id
balance_rial	bigint(20)	no		0	
updated_at	datetime(6)	no			

Foreign keys:

- fk_wallets_user → users (On delete RESTRICT)

Check constraints:

- ck_wallets_non_negative: balance_rial >= 0

13.1.7 wallet_transactions

The append-only wallet ledger with idempotency keys.

Column	Type	Null	Key	Default	References
id	bigint(20) unsigned	no	PRI	auto_increment	
user_id	bigint(20) unsigned	no	MUL		wallets.user_id
kind	varchar(16)	no	MUL		
amount_rial	bigint(20)	no			
balance_after_rial	bigint(20)	no			
reference_type	varchar(32)	yes			
reference_id	bigint(20) unsigned	yes			
description	varchar(255)	no			
idempotency_key	varchar(128)	yes			
created_by	bigint(20) unsigned	yes	MUL		users.id
created_at	datetime(6)	no			

Foreign keys:

- fk_wallet_transactions_creator → users (On delete RESTRICT)
- fk_wallet_transactions_wallet → wallets (On delete RESTRICT)

Check constraints:

- ck_wallet_transactions_amount: amount_rial <> 0
- ck_wallet_transactions_balance: balance_after_rial >= 0
- ck_wallet_transactions_kind: kind in ('topup','order_charge','usage','refund','adjustmen')

13.1.8 orders

Customer orders and their review.

Column	Type	Null	Key	Default	References
id	bigint(20) unsigned	no	PRI	auto_increment	
user_id	bigint(20) unsigned	no	MUL		users.id
status	varchar(16)	no	MUL		
customer_note	varchar(500)	yes			
upfront_rial	bigint(20) unsigned	no		0	
reviewer_id	bigint(20) unsigned	yes	MUL		users.id

Column	Type	Null	Key	Default	References
review_reason	varchar(500)	yes			
placed_at	datetime(6)	no			
reviewed_at	datetime(6)	yes			
updated_at	datetime(6)	no			

Foreign keys:

- `fk_orders_reviewer` → `users` (On delete RESTRICT)
- `fk_orders_user` → `users` (On delete RESTRICT)

Check constraints:

- `ck_orders_review`: `status` in ('pending','cancelled') or `reviewed_at` is not null
- `ck_orders_status`: `status` in ('pending','rejected','cancelled','provisioning','active','f

13.1.9 order_items

What an order asks for, with the prices copied at the time of ordering.

Column	Type	Null	Key	Default	References
id	bigint(20) unsigned	no	PRI	auto_increment	
order_id	bigint(20) unsigned	no	MUL		orders.id
plan_id	int(10) unsigned	no	MUL		plans.id
app_name	varchar(41)	no			
image	varchar(512)	no			
port	smallint(5) unsigned	no			
hourly_price_rial	bigint(20) unsigned	no			
monthly_price_rial	bigint(20) unsigned	no			

Foreign keys:

- `fk_order_items_order` → `orders` (On delete CASCADE)
- `fk_order_items_plan` → `plans` (On delete RESTRICT)

Check constraints:

- `ck_order_items_port`: `port` > 0

13.1.10 projects

Applications created by confirmed orders, with their billing state.

Column	Type	Null	Key	Default	References
id	bigint(20) unsigned	no	PRI	auto_increment	
user_id	bigint(20) unsigned	no	MUL		users.id
order_item_id	bigint(20) unsigned	no	UNI		order_items.id

Column	Type	Null	Key	Default	References
plan_id	int(10) unsigned	no	MUL		plans.id
app_name	varchar(41)	no	UNI		
status	varchar(16)	no	MUL		
server_id	varchar(64)	no			
command_id	char(36)	yes			
hourly_price_high	bigint(20) unsigned	no			
monthly_price_high	bigint(20) unsigned	no			
created_at	datetime(6)	no			
activated_at	datetime(6)	yes			
suspended_at	datetime(6)	yes			
billing_resumed_at	datetime(6)	yes			
deleted_at	datetime(6)	yes			
updated_at	datetime(6)	no			

Foreign keys:

- fk_projects_order_item → order_items (On delete RESTRICT)
- fk_projects_plan → plans (On delete RESTRICT)
- fk_projects_user → users (On delete RESTRICT)

Check constraints:

- ck_projects_status: status in ('provisioning','active','suspended','failed','deleted')

13.1.11 usage_records

One charged hour of one project; UNIQUE(project, hour) prevents double billing.

Column	Type	Null	Key	Default	References
id	bigint(20) unsigned	no	PRI	auto_increment	
project_id	bigint(20) unsigned	no	MUL		projects.id
period_start	datetime	no			
amount_rial	bigint(20) unsigned	no			
wallet_transaction_id	bigint(20) unsigned	yes	MUL		wallet_transactions.id
invoice_id	bigint(20) unsigned	yes	MUL		invoices.id
created_at	datetime(6)	no			

Foreign keys:

- fk_usage_records_invoice → invoices (On delete RESTRICT)
- fk_usage_records_project → projects (On delete RESTRICT)
- fk_usage_records_transaction → wallet_transactions (On delete RESTRICT)

13.1.12 invoices

Monthly, tax-inclusive invoices; total equals subtotal plus tax.

Column	Type	Null	Key	Default	References
id	bigint(20) unsigned	no	PRI	auto_increment	
number	varchar(24)	no	UNI		
user_id	bigint(20) unsigned	no	MUL		users.id
period_start	date	no			
period_end	date	no			
status	varchar(16)	no			
subtotal_rial	bigint(20) unsigned	no			
tax_rate_bp	smallint(5) unsigned	no			
tax_rial	bigint(20) unsigned	no			
total_rial	bigint(20) unsigned	no			
issued_at	datetime(6)	yes			
created_at	datetime(6)	no			

Foreign keys:

- `fk_invoices_user` → `users` (On delete RESTRICT)

Check constraints:

- `ck_invoices_period`: `period_end` >= `period_start`
- `ck_invoices_status`: `status` in ('draft','issued','paid','void')
- `ck_invoices_total`: `total_rial` = `subtotal_rial` + `tax_rial`

13.1.13 invoice_lines

One line per project per invoice: hours and amount.

Column	Type	Null	Key	Default	References
id	bigint(20) unsigned	no	PRI	auto_increment	
invoice_id	bigint(20) unsigned	no	MUL		invoices.id
project_id	bigint(20) unsigned	yes	MUL		projects.id
description	varchar(255)	no			
quantity_hours	int(10) unsigned	no			
amount_rial	bigint(20) unsigned	no			

Foreign keys:

- `fk_invoice_lines_invoice` → `invoices` (On delete CASCADE)
- `fk_invoice_lines_project` → `projects` (On delete SET NULL)

13.1.14 support_tickets

Customer support conversations.

Column	Type	Null	Key	Default	References
id	bigint(20) unsigned	no	PRI	auto_increment	
user_id	bigint(20) unsigned	no	MUL		users.id

Column	Type	Null	Key	Default	References
project_id	bigint(20) unsigned	yes	MUL		projects.id
subject	varchar(160)	no			
status	varchar(16)	no	MUL	'open'	
priority	varchar(8)	no		'normal'	
created_at	datetime(6)	no			
updated_at	datetime(6)	no			

Foreign keys:

- fk_support_tickets_project → projects (On delete SET NULL)
- fk_support_tickets_user → users (On delete RESTRICT)

Check constraints:

- ck_support_tickets_priority: priority in ('low','normal','high')
- ck_support_tickets_status: status in ('open','answered','closed')

13.1.15 ticket_messages

Messages within a support ticket.

Column	Type	Null	Key	Default	References
id	bigint(20) unsigned	no	PRI	auto_increment	
ticket_id	bigint(20) unsigned	no	MUL		support_tickets.id
author_id	bigint(20) unsigned	no	MUL		users.id
body	text	no			
created_at	datetime(6)	no			

Foreign keys:

- fk_ticket_messages_author → users (On delete RESTRICT)
- fk_ticket_messages_ticket → support_tickets (On delete CASCADE)

13.2 Platform tables

13.2.1 agents

Enrolled pull agents and their certificate serials.

Column	Type	Null	Key	Default	References
id	varchar(64)	no	PRI		
name	varchar(96)	no			
certificate_expires_at	datetime(6)	no			

13.2.2 agent_ca

The single certificate authority that signs machine-agent client certificates; its key is sealed.

Column	Type	Null	Key	Default	References
id	tinyint(3) unsigned	no	PRI		
certificate_pem	text	no			
private_key_sealed	varbinary(4096)	no			
created_at	datetime(6)	no			

Check constraints:

- ck_agent_ca_singleton: id = 1

13.2.3 agent_claims

Install-first claim codes an agent presents before an administrator approves it.

Column	Type	Null	Key	Default	References
id	varchar(64)	no	PRI		
name	varchar(96)	no			
csr	text	no			
code_digest	char(64)	no	UNI		
secret_digest	char(64)	no			
expires_at	datetime(6)	no	MUL		
enrollment_agent_id	varchar(64)	yes			
enrollment_authority_epoch	bigint(20) unsigned	yes			
enrollment_certificate	text	yes			
enrollment_expires_at	datetime(6)	yes			
position	int(10) unsigned	no			

13.2.4 agent_enrollment_tokens

One-time enrollment tokens (stored as hashes) for dashboard-generated install commands.

Column	Type	Null	Key	Default	References
code_digest	char(64)	no	PRI		
name	varchar(96)	yes			
expires_at	datetime(6)	no	MUL		

13.2.5 applications

Applications SwarmOps renders and deploys, with their image, port and resource limits.

Column	Type	Null	Key	Default	References
name	varchar(41)	no	PRI		
image	varchar(512)	no			
port	smallint(5) unsigned	no			
replicas	int(10) unsigned	no			
cpus	double	no			
memory_mib	bigint(20)	no			
plan	varchar(32)	yes			
domain	varchar(253)	yes	UNI		

Column	Type	Null	Key	Default	References
resolver	varchar(64)	yes			
backend	varchar(41)	yes			
database_delivery	varchar(32)	yes			
health_path	varchar(201)	yes			
metrics	tinyint(1)	no		0	
metrics_path	varchar(201)	yes			
metrics_port	smallint(5) unsigned	yes			
tracing	tinyint(1)	no		0	
created_at	datetime(6)	no			
updated_at	datetime(6)	no			

Check constraints:

- ck_applications_resources: cpus > 0 and memory_mib > 0 and port > 0

13.2.6 application_databases

Managed database engines an application depends on.

Column	Type	Null	Key	Default	References
application_name	varchar(41)	no	PRI		applications.name
engine	varchar(16)	no	PRI		
position	smallint(5) unsigned	no			

Foreign keys:

- fk_application_databases_application → applications (On delete CASCADE)

13.2.7 application_database_credentials

Sealed credentials an application uses for each managed database engine.

Column	Type	Null	Key	Default	References
application_name	varchar(41)	no	PRI		
engine	varchar(16)	no	PRI		
uri_sealed	varbinary(4096)	no			
updated_at	datetime(6)	no			

13.2.8 application_database_env

Environment variable names through which database credentials reach an application.

Column	Type	Null	Key	Default	References
application_name	varchar(41)	no	PRI		applications.name
engine	varchar(16)	no	PRI		
position	smallint(5) unsigned	no	PRI		

Column	Type	Null	Key	Default	References
env_name	varchar(64)	no			

Foreign keys:

- fk_application_database_env_application → applications (On delete CASCADE)

13.2.9 application_env

An application's environment variables; values are sealed.

Column	Type	Null	Key	Default	References
application_name	varchar(41)	no	PRI		applications.name
env_key	varchar(64)	no	PRI		
env_value_sealed	varbinary(8192)	no			

Foreign keys:

- fk_application_env_application → applications (On delete CASCADE)

13.2.10 application_health_command

The ordered arguments of an application's custom health command.

Column	Type	Null	Key	Default	References
application_name	varchar(41)	no	PRI		applications.name
position	smallint(5) unsigned	no	PRI		
argument	varchar(1024)	no			

Foreign keys:

- fk_application_health_command_application → applications (On delete CASCADE)

13.2.11 application_outcomes

The latest start outcome of each application.

Column	Type	Null	Key	Default	References
application_name	varchar(41)	no	PRI		applications.name
started	tinyint(1)	no		0	
started_at	datetime(6)	yes			
last_attempt_at	datetime(6)	yes			
last_command_id	varchar(64)	yes			
failure_summary	text	yes			

Foreign keys:

- fk_application_outcomes_application → applications (On delete CASCADE)

13.2.12 audit_events

Append-only record of every operator and customer action.

Column	Type	Null	Key	Default	References
seq	bigint(20) unsigned	no	PRI	auto_increment	
id	char(32)	no	UNI		
occurred_at	datetime(6)	no	MUL		
actor	varchar(190)	no	MUL		
action	varchar(190)	no			
target	varchar(512)	no			
outcome	varchar(64)	no			
request_id	varchar(128)	yes			

13.2.13 audit_event_details

Key-value details of an audit event.

Column	Type	Null	Key	Default	References
event_seq	bigint(20) unsigned	no	PRI		audit_events.seq
detail_key	varchar(190)	no	PRI		
detail_value	text	no			

Foreign keys:

- fk_audit_event_details_event → audit_events (On delete CASCADE)

13.2.14 commands

The durable command queue: action, target, state, attempts, leases and failure details.

Column	Type	Null	Key	Default	References
seq	bigint(20) unsigned	no	UNI	auto_increment	
id	char(36)	no	PRI		
action	varchar(64)	no	MUL		
actor	varchar(190)	no	MUL		
server_id	varchar(64)	no	MUL		
node_id	varchar(64)	no		''	
cluster_id	varchar(64)	no			
target	varchar(512)	no			
state	varchar(20)	no	MUL		
attempt	int(10) unsigned	no		0	
max_attempts	tinyint(3) unsigned	no			
auto_retry	tinyint(1)	no			
authority_epoch	bigint(20) unsigned	no			
request_id	varchar(128)	yes			
idempotency_key	varchar(128)	no			
payload_digest	char(64)	yes			
has_artifact	tinyint(1)	no		0	

Column	Type	Null	Key	Default	References
lease_id	varchar(64)	yes			
lease_expires_at	datetime(6)	yes			
last_attempt_at	datetime(6)	yes			
next_attempt_at	datetime(6)	yes			
last_error	text	yes			
failure_code	varchar(64)	yes			
failure_summary	text	yes			
recovery_hint	text	yes			
created_at	datetime(6)	no			
updated_at	datetime(6)	no	MUL		

Check constraints:

- ck_commands_attempts: max_attempts between 1 and 8
- ck_commands_epoch: authority_epoch >= 1
- ck_commands_state: state in ('uploading','queued','leased','preparing','running','retry_')

13.2.15 command_events

Timeline of state changes and evidence for each command.

Column	Type	Null	Key	Default	References
command_id	char(36)	no	PRI		commands.id
sequence	int(10) unsigned	no	PRI		
state	varchar(20)	no			
evidence	varchar(1024)	yes			
occurred_at	datetime(6)	no			

Foreign keys:

- fk_command_events_command → commands (On delete CASCADE)

13.2.16 command_locks

Named locks serialising commands that must not run concurrently.

Column	Type	Null	Key	Default	References
lock_name	varchar(80)	no	PRI		

13.2.17 command_outputs

Sealed output streams captured from a command's execution.

Column	Type	Null	Key	Default	References
command_id	char(36)	no	PRI		commands.id
output_sealed	mediumblob	no			
retained_at	datetime(6)	no			

Foreign keys:

- `fk_command_outputs_command` → `commands` (On delete CASCADE)

13.2.18 `command_payloads`

Sealed input payload of a command.

Column	Type	Null	Key	Default	References
<code>command_id</code>	<code>char(36)</code>	no	PRI		<code>commands.id</code>
<code>payload_sealed</code>	<code>mediumblob</code>	no			

Foreign keys:

- `fk_command_payloads_command` → `commands` (On delete CASCADE)

13.2.19 `core_authority`

The single row holding the authority epoch and the active core.

Column	Type	Null	Key	Default	References
<code>id</code>	<code>tinyint(3) unsigned</code>	no	PRI		
<code>active_id</code>	<code>varchar(64)</code>	yes			
<code>authority_epoch</code>	<code>bigint(20) unsigned</code>	no			
<code>updated_at</code>	<code>datetime(6)</code>	no			

Check constraints:

- `ck_core_authority_epoch`: `authority_epoch >= 1`
- `ck_core_authority_singleton`: `id = 1`

13.2.20 `core_handoffs`

Planned movements of authority between core members.

Column	Type	Null	Key	Default	References
<code>id</code>	<code>tinyint(3) unsigned</code>	no	PRI		
<code>from_id</code>	<code>varchar(64)</code>	no	MUL		<code>core_members.id</code>
<code>to_id</code>	<code>varchar(64)</code>	no	MUL		<code>core_members.id</code>
<code>state</code>	<code>varchar(16)</code>	no			
<code>prepared_at</code>	<code>datetime(6)</code>	no			
<code>fenced_at</code>	<code>datetime(6)</code>	yes			

Foreign keys:

- `fk_core_handoffs_from` → `core_members` (On delete RESTRICT)
- `fk_core_handoffs_to` → `core_members` (On delete RESTRICT)

Check constraints:

- `ck_core_handoffs_distinct`: `from_id <> to_id`
- `ck_core_handoffs_fenced_time`: `state <> 'fenced'` or `fenced_at` is not null
- `ck_core_handoffs_singleton`: `id = 1`
- `ck_core_handoffs_state`: `state in ('prepared', 'fenced')`

13.2.21 core_members

Known core instances and their roles.

Column	Type	Null	Key	Default	References
id	varchar(64)	no	PRI		
name	varchar(96)	no			
endpoint	varchar(512)	no		''	
role	varchar(16)	no			
replica_state	varchar(32)	no			
agent_server_id	varchar(64)	yes			
last_checkpoint_at	datetime(6)	yes			
position	smallint(5) unsigned	no			

Check constraints:

- ck_core_members_replica_state: replica_state in ('awaiting_restore','verified')
- ck_core_members_role: role in ('active','standby')

13.2.22 database_credentials

Sealed administrative credentials of managed database engines.

Column	Type	Null	Key	Default	References
engine	varchar(16)	no	PRI		
uri_sealed	varbinary(4096)	no			
updated_at	datetime(6)	no			

13.2.23 dependency_bindings

Declared network dependencies between routed services.

Column	Type	Null	Key	Default	References
cluster_id	varchar(64)	no	PRI		routing_clusters.cluster_id
caller_service	varchar(128)	no	PRI		
target_route	varchar(64)	no	PRI		
name	varchar(64)	no	PRI	''	
delivery	varchar(32)	no			
version	int(11)	no			

Foreign keys:

- fk_dependency_bindings_cluster → routing_clusters (On delete CASCADE)

13.2.24 dns_credential_versions

Versions of DNS provider credentials; secrets are sealed.

Column	Type	Null	Key	Default	References
cluster_id	varchar(64)	no	PRI		routing_clusters.cluster_id

Column	Type	Null	Key	Default	References
credential_id	varchar(64)	no	PRI		
version	int(10) unsigned	no	PRI		
name	varchar(96)	no			
provider	varchar(32)	no			
account_id	varchar(128)	yes			
email	varchar(254)	yes			
secret_name	varchar(128)	no			
state	varchar(16)	no			
created_at	datetime(6)	no			
validated_at	datetime(6)	yes			
secret_sealed	varbinary(8192)	yes			

Foreign keys:

- `fk_dns_credential_versions_cluster` → `routing_clusters` (On delete CASCADE)

Check constraints:

- `ck_dns_credential_versions_secret`: `state = 'removed'` or `secret_sealed` is not null
- `ck_dns_credential_versions_state`: `state` in ('sealed','validated','removed')

13.2.25 dns_records

DNS records SwarmOps manages at providers.

Column	Type	Null	Key	Default	References
cluster_id	varchar(64)	no	PRI		<code>routing_clusters.cluster_id</code>
id	varchar(64)	no	PRI		
zone	varchar(253)	no			
name	varchar(253)	no			
type	varchar(8)	no			
content	varchar(512)	no			
ttl	int(10) unsigned	no			
proxied	tinyint(1)	no			
managed	tinyint(1)	no			
adopted	tinyint(1)	no			
credential_id	varchar(64)	no			
provider_record_id	varchar(128)	yes			
version	int(11)	no			

Foreign keys:

- `fk_dns_records_cluster` → `routing_clusters` (On delete CASCADE)

13.2.26 routes

Declared routes: protocol, scope, TLS, target port and health check.

Column	Type	Null	Key	Default	References
cluster_id	varchar(64)	no	PRI		routing_clusters.cluster_id
route_key	varchar(64)	no	PRI		
service_key	varchar(128)	no			
protocol	varchar(8)	no			
scope	varchar(16)	no			
tls_mode	varchar(16)	no			
listen_port	smallint(5) unsigned	yes			
target_port	smallint(5) unsigned	no			
path_prefix	varchar(256)	yes			
health_kind	varchar(32)	no		''	
health_path	varchar(256)	yes			
health_timeout_seconds	smallint(5) unsigned	no		0	
dns_reference	varchar(64)	yes			
resolver	varchar(64)	yes			
enabled	tinyint(1)	no			
managed	tinyint(1)	no			
metrics	tinyint(1)	no			
access_logs	tinyint(1)	no			
public_allow	tinyint(1)	no			
is_sensitive	tinyint(1)	no			
version	int(11)	no			

Foreign keys:

- fk_routes_cluster → routing_clusters (On delete CASCADE)

Check constraints:

- ck_routes_protocol: protocol in ('http','tcp','udp')

13.2.27 route_certificates

Certificates observed on the gateway.

Column	Type	Null	Key	Default	References
cluster_id	varchar(64)	no	PRI		routes.cluster_id
route_key	varchar(64)	no	PRI		routes.route_key
state	varchar(32)	no			
issuer	varchar(256)	yes			
fingerprint	varchar(128)	yes			
resolver	varchar(64)	no		''	
handshake_valid	tinyint(1)	no			
failure_summary	text	yes			
last_attempt	datetime(6)	yes			
not_before	datetime(6)	yes			
not_after	datetime(6)	yes	MUL		
version	int(11)	no			

Foreign keys:

- `fk_route_certificates_route` → `routes` (On delete CASCADE)

13.2.28 route_certificate_domains

Domains covered by a gateway certificate.

Column	Type	Null	Key	Default	References
<code>cluster_id</code>	<code>varchar(64)</code>	no	PRI		<code>route_certificates.cluster_id</code>
<code>route_key</code>	<code>varchar(64)</code>	no	PRI		<code>route_certificates.route_key</code>
<code>position</code>	<code>smallint(5) unsigned</code>	no	PRI		
<code>domain</code>	<code>varchar(253)</code>	no			

Foreign keys:

- `fk_route_certificate_domains_certificate` → `route_certificates` (On delete CASCADE)

13.2.29 route_hosts

Host names and aliases of a route.

Column	Type	Null	Key	Default	References
<code>cluster_id</code>	<code>varchar(64)</code>	no	PRI		<code>routes.cluster_id</code>
<code>route_key</code>	<code>varchar(64)</code>	no	PRI		<code>routes.route_key</code>
<code>kind</code>	<code>varchar(4)</code>	no	PRI		
<code>position</code>	<code>smallint(5) unsigned</code>	no	PRI		
<code>hostname</code>	<code>varchar(253)</code>	no			

Foreign keys:

- `fk_route_hosts_route` → `routes` (On delete CASCADE)

Check constraints:

- `ck_route_hosts_kind`: `kind` in ('host','sni')

13.2.30 route_runtime

The gateway's observed runtime state of a route.

Column	Type	Null	Key	Default	References
<code>cluster_id</code>	<code>varchar(64)</code>	no	PRI		<code>routing_clusters.cluster_id</code>
<code>route_key</code>	<code>varchar(64)</code>	no	PRI		
<code>protocol</code>	<code>varchar(8)</code>	no			
<code>router</code>	<code>varchar(256)</code>	no		''	
<code>service</code>	<code>varchar(256)</code>	no		''	
<code>state</code>	<code>varchar(32)</code>	no		''	
<code>observed_at</code>	<code>datetime(6)</code>	no			
<code>version</code>	<code>int(11)</code>	no			

Foreign keys:

- `fk_route_runtime_cluster` → `routing_clusters` (On delete CASCADE)

13.2.31 `route_runtime_entry_points`

Entry points a route is observed on.

Column	Type	Null	Key	Default	References
<code>cluster_id</code>	<code>varchar(64)</code>	no	PRI		<code>route_runtime.cluster_id</code>
<code>route_key</code>	<code>varchar(64)</code>	no	PRI		<code>route_runtime.route_key</code>
<code>position</code>	<code>smallint(5)</code> <code>unsigned</code>	no	PRI		
<code>entry_point</code>	<code>varchar(64)</code>	no			

Foreign keys:

- `fk_route_runtime_entry_points_runtime` → `route_runtime` (On delete CASCADE)

13.2.32 `route_runtime_errors`

Errors the gateway reports for a route.

Column	Type	Null	Key	Default	References
<code>cluster_id</code>	<code>varchar(64)</code>	no	PRI		<code>route_runtime.cluster_id</code>
<code>route_key</code>	<code>varchar(64)</code>	no	PRI		<code>route_runtime.route_key</code>
<code>position</code>	<code>smallint(5)</code> <code>unsigned</code>	no	PRI		
<code>message</code>	<code>varchar(256)</code>	no			

Foreign keys:

- `fk_route_runtime_errors_runtime` → `route_runtime` (On delete CASCADE)

13.2.33 `routing_clusters`

Per-cluster gateway settings and cutover plans (JSON documents).

Column	Type	Null	Key	Default	References
<code>cluster_id</code>	<code>varchar(64)</code>	no	PRI		
<code>settings_json</code>	<code>longtext</code>	no			
<code>cutover_json</code>	<code>longtext</code>	yes			
<code>cutover_rollback_json</code>	<code>longtext</code>	yes			
<code>updated_at</code>	<code>datetime(6)</code>	no			

Check constraints:

- `ck_routing_clusters_settings`: `json_valid(settings_json)`
- `cutover_json`: `json_valid(cutover_json)`
- `cutover_rollback_json`: `json_valid(cutover_rollback_json)`
- `settings_json`: `json_valid(settings_json)`

13.2.34 routing_domains

Domains a cluster may route.

Column	Type	Null	Key	Default	References
cluster_id	varchar(64)	no	PRI		routing_clusters.cluster_id
zone	varchar(253)	no	PRI		
note	varchar(512)	yes			
created_at	datetime(6)	yes			
version	int(11)	no			

Foreign keys:

- fk_routing_domains_cluster → routing_clusters (On delete CASCADE)

13.2.35 schema_migrations

Applied schema migrations with their checksums.

Column	Type	Null	Key	Default	References
version	int(10) unsigned	no	PRI		
name	varchar(200)	no			
checksum	char(64)	no			
applied_at	datetime(6)	no			

13.2.36 servers

Saved servers, their connection details and last observed agent and Swarm health.

Column	Type	Null	Key	Default	References
id	varchar(64)	no	PRI		
name	varchar(96)	no	MUL		
host	varchar(253)	no			
port	smallint(5) unsigned	no		0	
username	varchar(190)	no		''	
authentication	varchar(32)	no			
connection_type	varchar(32)	yes			
api_url	varchar(512)	yes			
host_key_fingerprint	varchar(128)	no		''	
tls_certificate_fingerprint	varchar(128)	yes			
docker_available	tinyint(1)	no		0	
docker_version	varchar(64)	yes			
swarm_control_available	tinyint(1)	no		0	
swarm_state	varchar(32)	yes			
last_connected_at	datetime(6)	yes			
agent_version	varchar(64)	yes			
agent_checked_at	datetime(6)	yes			
agent_detail	text	yes			
agent_last_failure_at	datetime(6)	yes			
agent_last_reachable_at	datetime(6)	yes			

Column	Type	Null	Key	Default	References
agent_protocol_version	int(10) unsigned	no		0	
agent_state	varchar(16)	yes			
agent_summary	varchar(1024)	yes			
agent_uptime_seconds	bigint(20) unsigned	no		0	
update_automatic	tinyint(1)	no		0	
update_checked_at	datetime(6)	yes			
update_last_updated_at	datetime(6)	yes			
update_requested_at	datetime(6)	yes			
update_revision	varchar(128)	yes			
update_state	varchar(32)	yes			
update_version	varchar(64)	yes			
updated_at	datetime(6)	no			

Check constraints:

- ck_servers_agent_state: agent_state is null or agent_state in ('healthy','degraded','unk

13.2.37 server_agent_events

History of machine-agent health observations per server.

Column	Type	Null	Key	Default	References
server_id	varchar(64)	no	PRI		servers.id
position	smallint(5) unsigned	no	PRI		
code	varchar(64)	no			
level	varchar(16)	no			
message	text	no			
occurred_at	datetime(6)	no			
source	varchar(32)	no			

Foreign keys:

- fk_server_agent_events_server → servers (On delete CASCADE)

13.2.38 server_keys

Sealed SSH or API keys of saved servers.

Column	Type	Null	Key	Default	References
server_id	varchar(64)	no	PRI		servers.id
api_key_sealed	varbinary(1024)	no			
updated_at	datetime(6)	no			

Foreign keys:

- fk_server_keys_server → servers (On delete CASCADE)

13.2.39 service_route_declarations

How each service participates in routing.

Column	Type	Null	Key	Default	References
cluster_id	varchar(64)	no	PRI		routing_clusters.cluster_id
service_key	varchar(128)	no	PRI		
role	varchar(32)	no			
reason	varchar(512)	yes			
version	int(11)	no			

Foreign keys:

- fk_service_route_declarations_cluster → routing_clusters (On delete CASCADE)

13.2.40 source_connections

Git provider connections; tokens are sealed.

Column	Type	Null	Key	Default	References
id	char(32)	no	PRI		
kind	varchar(32)	no			
name	varchar(190)	no	MUL		
base_url	varchar(512)	no			
account	varchar(190)	yes			
token_sealed	varbinary(8192)	no			
created_at	datetime(6)	no			
updated_at	datetime(6)	no			

13.2.41 source_private_hosts

Private Git hosts allowed for source deployments.

Column	Type	Null	Key	Default	References
host	varchar(253)	no	PRI		
position	smallint(5) unsigned	no	UNI		

13.2.42 source_settings

The single row of source-to-deploy settings.

Column	Type	Null	Key	Default	References
id	tinyint(3) unsigned	no	PRI		
enabled	tinyint(1)	no			
build_enabled	tinyint(1)	no			
image_prefix	varchar(255)	no		''	
registry_server	varchar(253)	no		''	
registry_username	varchar(190)	no		''	
registry_password_sealed	varbinary(4096)	yes			
updated_at	datetime(6)	no			

Column	Type	Null	Key	Default	References
--------	------	------	-----	---------	------------

Check constraints:

- `ck_source_settings_build_requires_enabled`: `build_enabled = 0` or `enabled = 1`
- `ck_source_settings_singleton`: `id = 1`